One Engineering System

Microsoft

# How Microsoft is transforming our software development culture

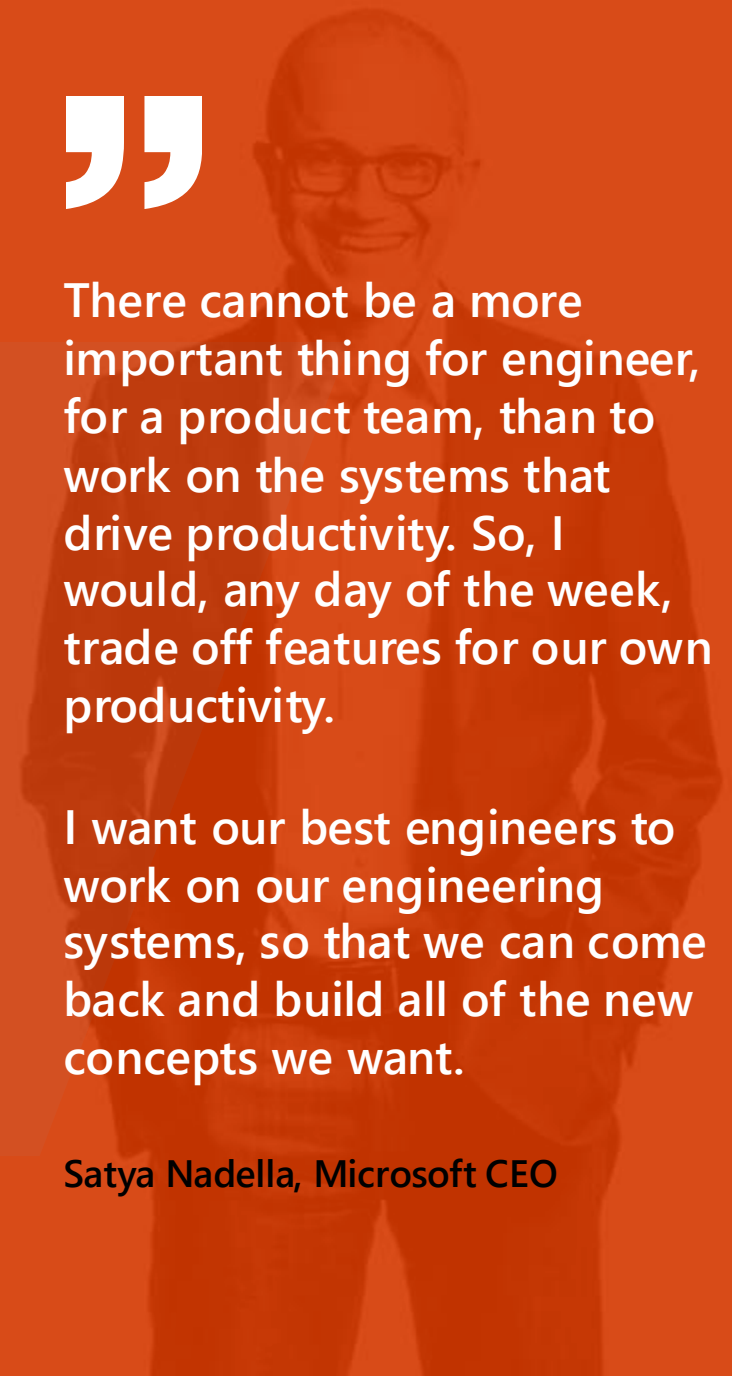
Jay Schmelzer
Partner Director of Product
Developer Division, Microsoft

# Enter the 1ES mission

Enable Microsoft teams with world-class tools and systems that help them ship products their customers love.

## Aligning to shared objectives:

- Create a community across the company to help drive tooling, culture, and process learnings
- Creating integrated and connected services promoting reuse both internally and externally
- Building privacy, security, and accessibility standards into our workflow



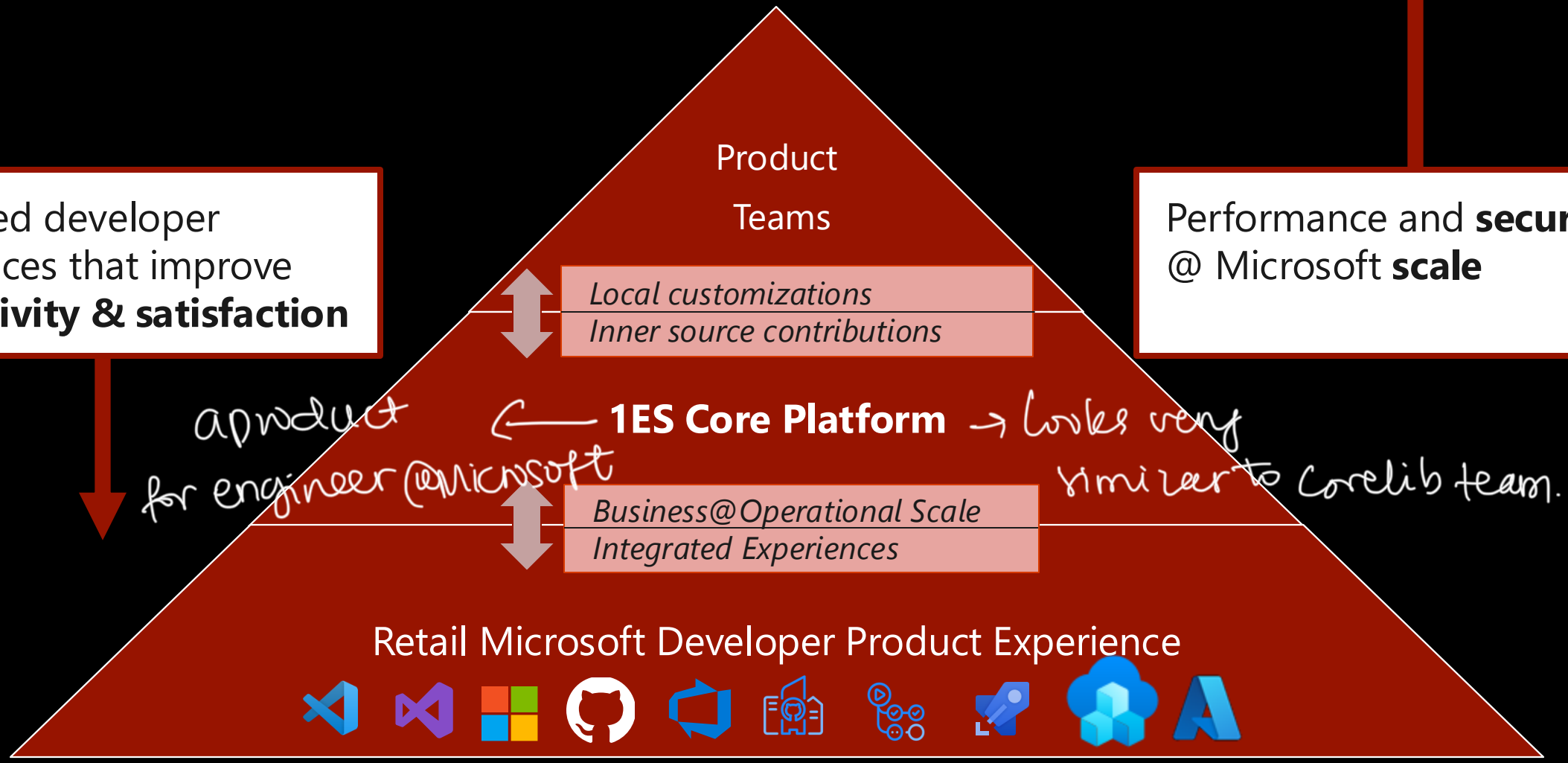
> There cannot be a more important thing for engineer, for a product team, than to work on the systems that drive productivity. So, I would, any day of the week, trade off features for our own productivity.
>
> I want our best engineers to work on our engineering systems, so that we can come back and build all of the new concepts we want.

Satya Nadella, Microsoft CEO

# Our 1ES Solution Approach

Integrated developer experiences that improve **productivity & satisfaction**

Performance and **security** @ Microsoft **scale**



# Microsoft engineering by the numbers

**99K**
Active Repos

**179K**
Monthly Engaged Users (MEU)

**2M**
Work Items Create per Month

**20M**
Builds per Month

**817TB**
Source Code

**350GB**
Largest Git Repo

**605K**
Pull Requests Completed per Month

**719K**
Average daily compute hours of Test

# What do we want to improve?

Speed & Innovation

Talent & Happiness

ask developers.


Trust & Quality

Collaboration & Scale

# How do we measure success?

Code velocity
(DORA Metrics)

+ Time to set-up env to start the work?
+ Time to 1st PR?

MTTR* for incidents,
security & compliance bugs


ask dev for feedback.

NSAT* surveys
Critical dev workflow times

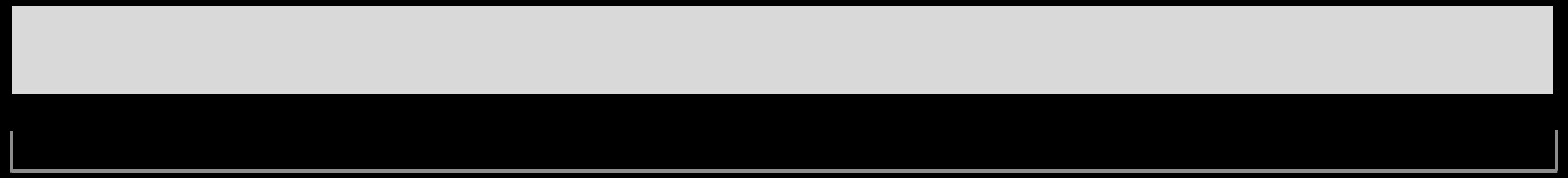
Engineering Thrive
- how their team is doing to strive for improvement
- not a tool for any assessments
- only shared for info purpose.

Opensource use
Innersource contributions
Manual steps for workflows

*MTTR = Mean-time-to-remediate
*NSAT = Net satisfaction score

# Before

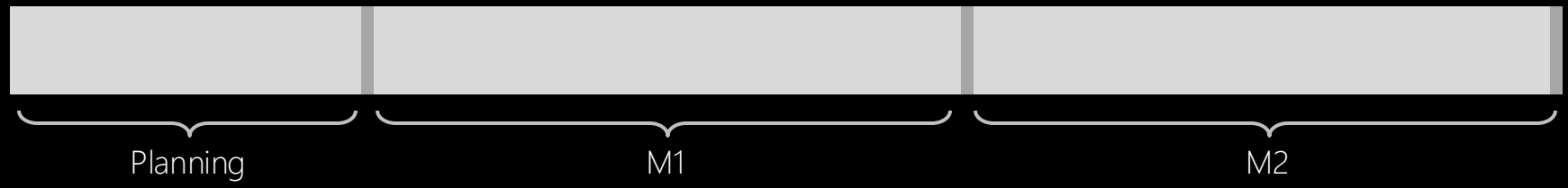
The OLD way



2 years

# Before

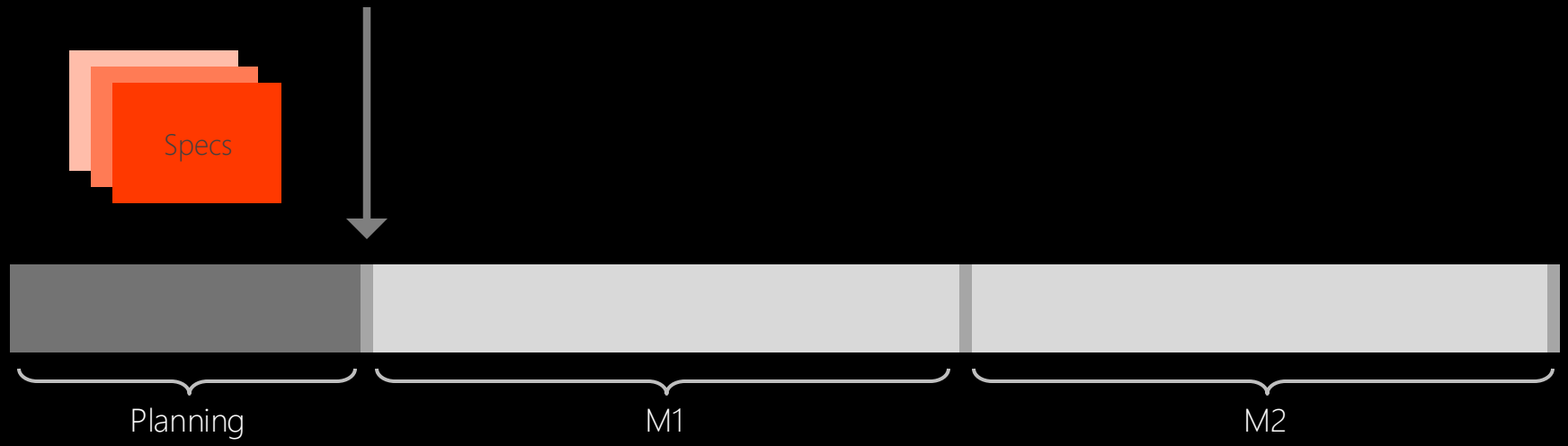
The OLD way



# Before

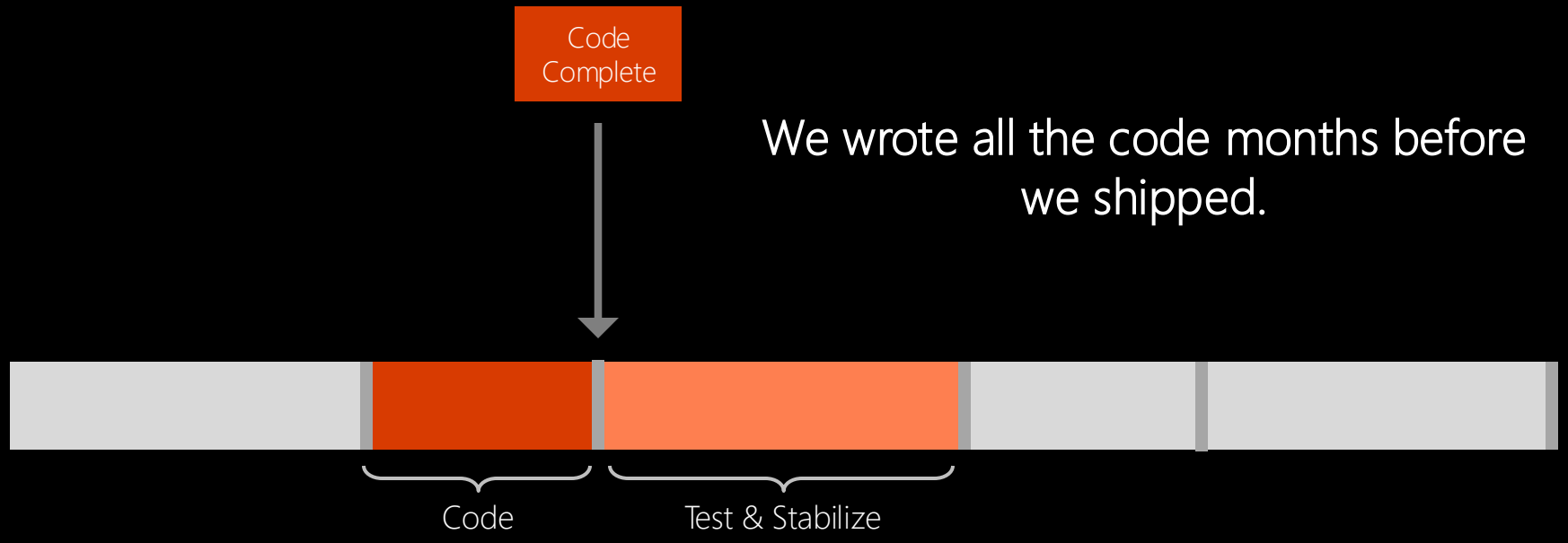
The OLD way

We knew exactly what to build…
and we knew it was right!



# Before

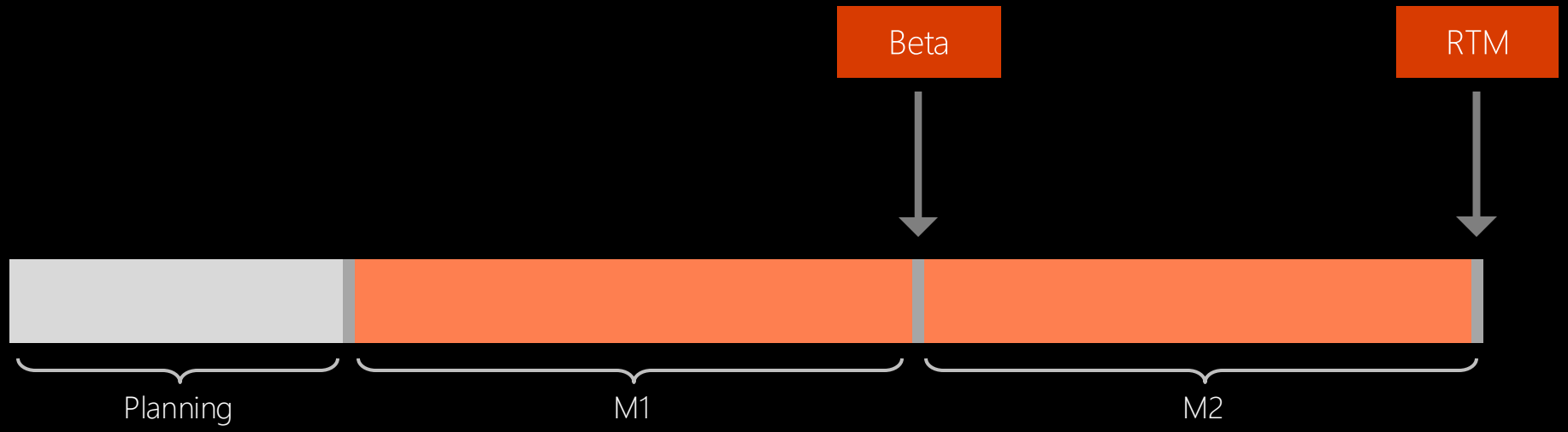
The OLD way



# Before

The OLD way

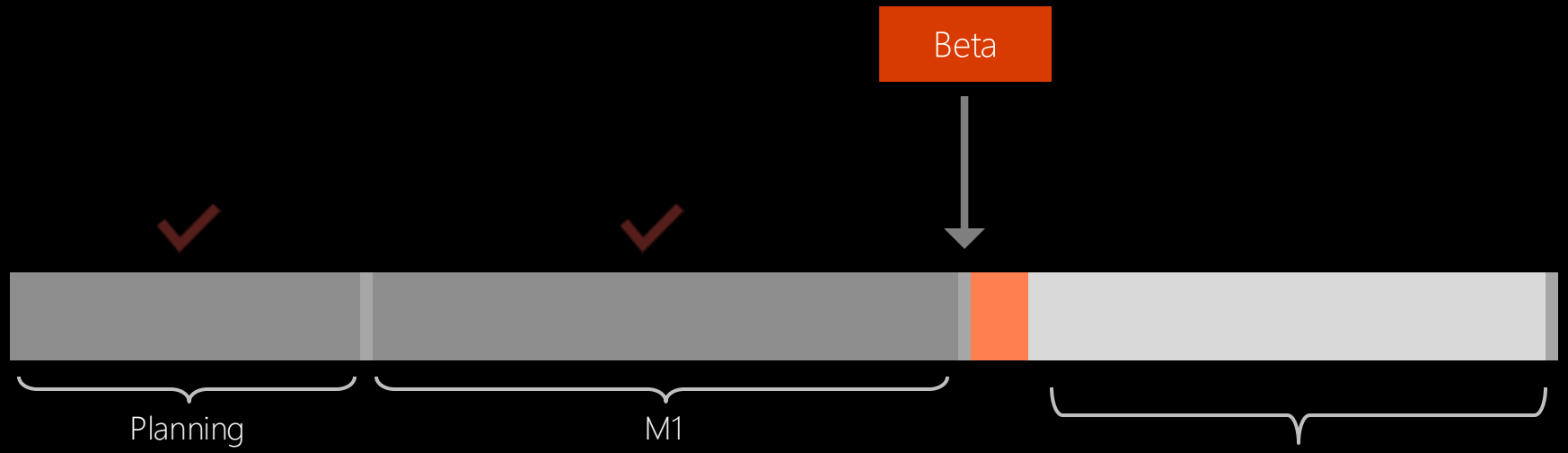
We had a perfect schedule and knew exactly when it would be ready!



# Before

The OLD way

Customer feedback – we should change the way a feature works. We didn't get it *quite* right...



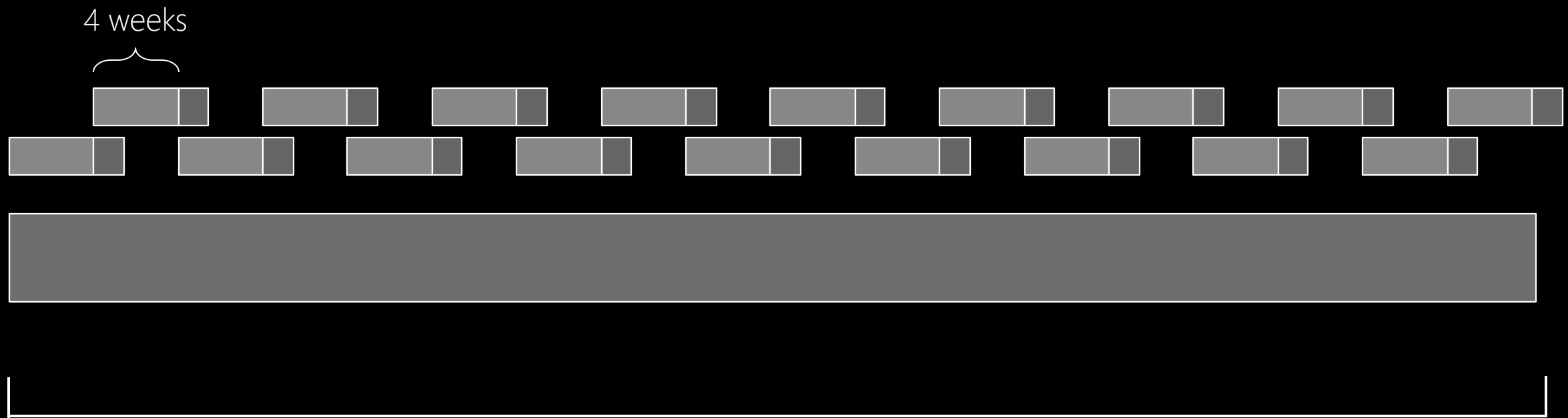

... but we're booked solid already.

# Before

The OLD way

*"Great feedback. Thanks! We'll take a look in planning for the next release. We should get it to you....*
*in a few years."*

# Now



# Before

4-6 month milestones
Horizontal teams
Personal offices
Long planning cycles
PM, Dev, Test
Yearly customer engagement
Feature branches
20+ person teams
Secret roadmap
Mountains of Bug debt
100 page spec documents
Deep organizational hierarchy
Success is a measure of install numbers
No concept of work in progress (WIP)
Features shipped once a year

# After

4-week sprints
Vertical teams
Team rooms
Continual Planning & Learning
PM & Engineering
Continual customer engagement
Everyone in master
8-12 person teams
Publicly shared roadmap
Minimal debt
Specs in PPT
Flattened organization hierarchy
User satisfaction determines success
WIP Limits measured
Features shipped every sprint

Microsoft

# Cultural & Org Transformation

# Cultural & Organization Transformation

- Create Clarity
- Be Customer Obsessed
- Evolve Full Stack Teams

Create clarity

70% can be good as well when the outcomes is good.

# Measure outcomes not outputs

## Objectives and key results (OKRs)

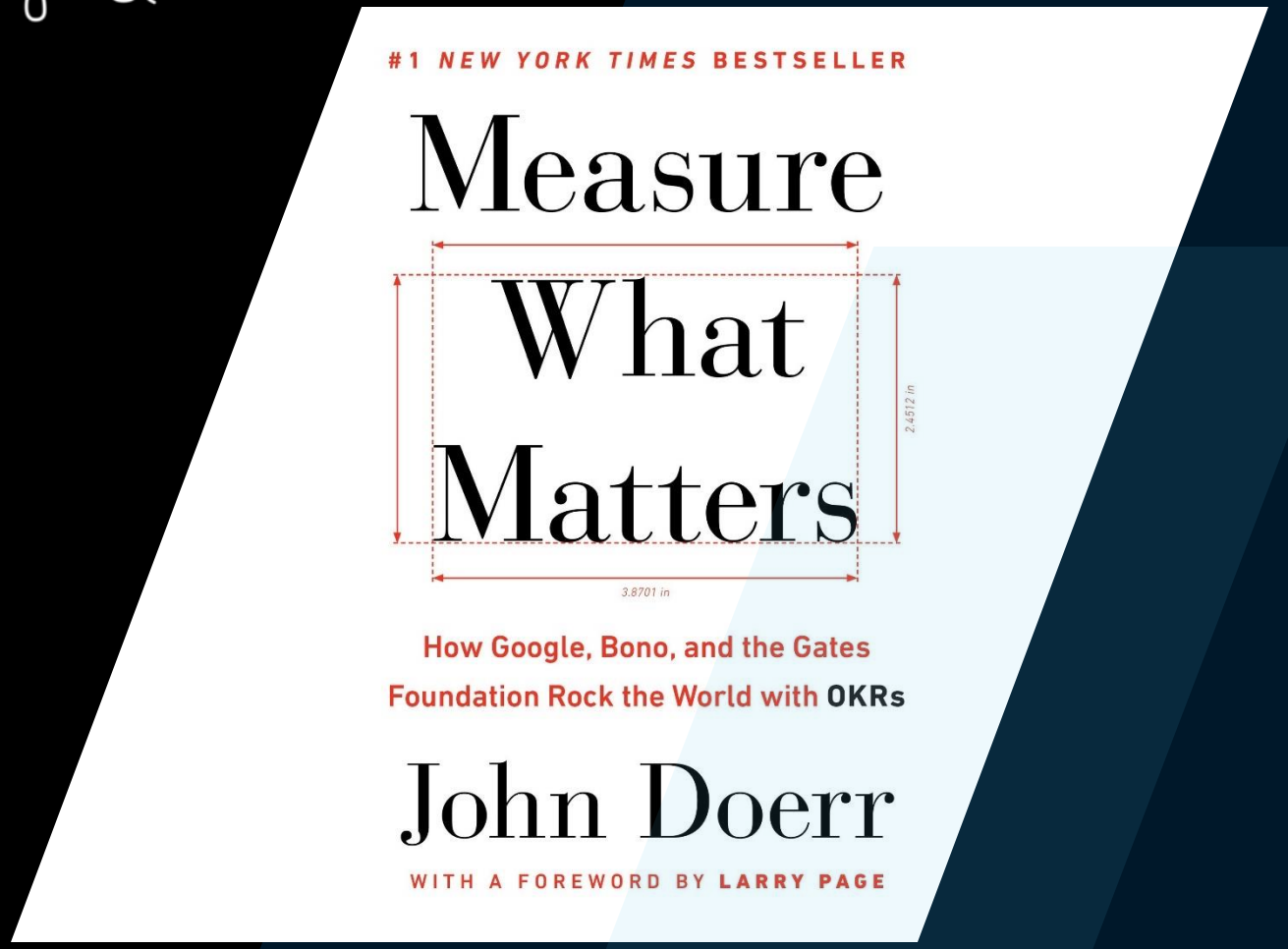
**Example objective: Grow a strong and happy customer base**

1. Increase external NPS from 21 to 35
2. Increase docs SAT from 55 to 65
3. New pipeline flow has an Apdex score of 0.9
4. Queue time for jobs is 5 seconds or less

**KRs are measures for the quarter.**

**Best KRs are leading indicators.**

**Encourage ambitious KRs: 70% of the improvement target scores green.**




Measure business outcomes with objectives and key results (OKRs) - Cloud Adoption Framework | Microsoft Docs

Create clarity

# Eliminate unhelpful KPIs

Here are a list of things we don't watch:

- Original estimate
- Completed hours
- Lines of code
- Team capacity
- Team burndown
- Team velocity
- # of bugs found — sandbagging
- % code coverage

Create clarity

# Change what you track

## Focus on measuring only the most critical and impactful KPIs:

measure engage users (who actively using the product)

### Customer usage

How much value are users getting?

- Acquisition
- Retention
- Engagement
- Satisfaction (NPS)
- Feature usage

### Pipeline throughput

How efficient is the DevOps process?

- Time to build
- Time to test
- Time to deploy
- Time to improve
- Failed and flaky automation

→ how much time spent on getting things done

### Live-site health

How quickly can you detect and fix issues?

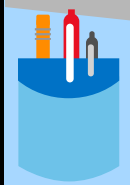
- Time to detect, time to communicate, time to mitigate
- Customer impact, customer support metrics
- Incident prevention items
- Aging live-site problems
- SLA per customer

Be customer obsessed

# Hypothesis based development

the manager still held accountable.

## Hypothesis

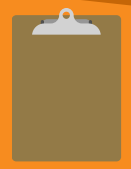


We believe {customer segment} wants {product/feature} because {value prop}

## Experiment

To prove or disprove the above, the team will conduct the following experiment(s): …

## Learning

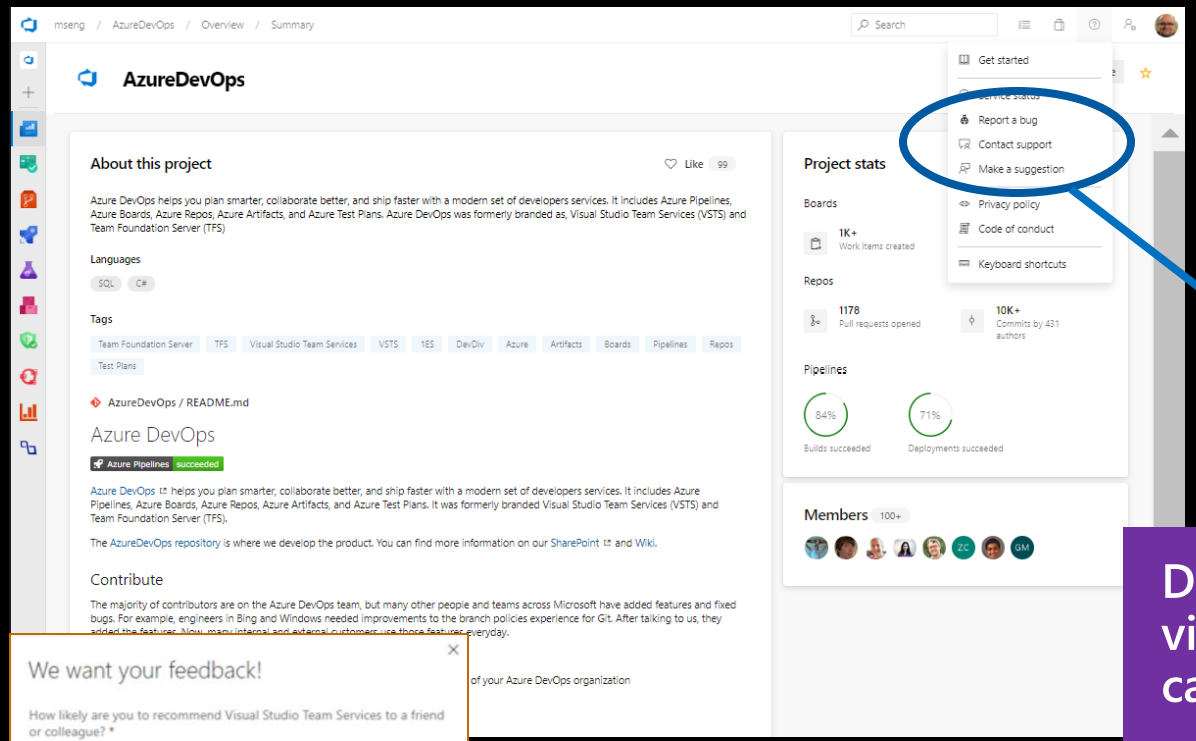


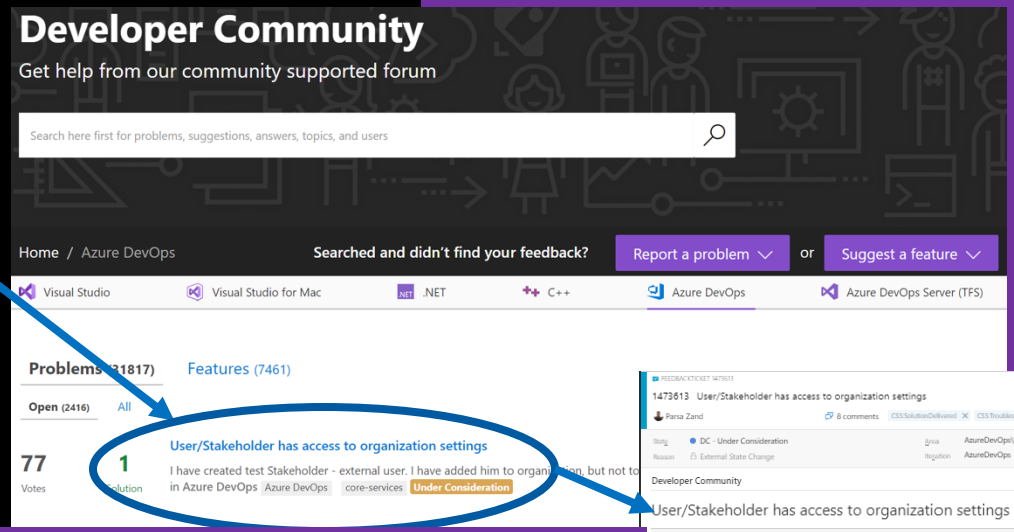
The above experiment(s) proved or disproved the hypothesis by impacting the following metric(s): …

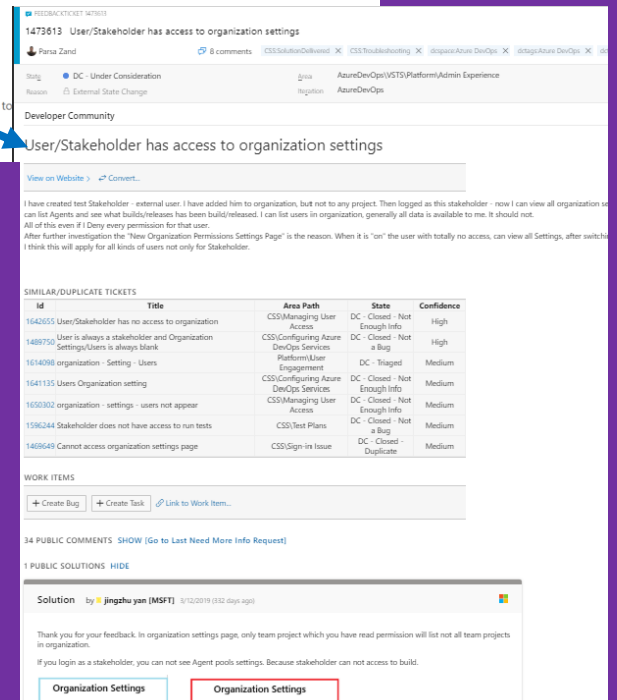
Be customer obsessed

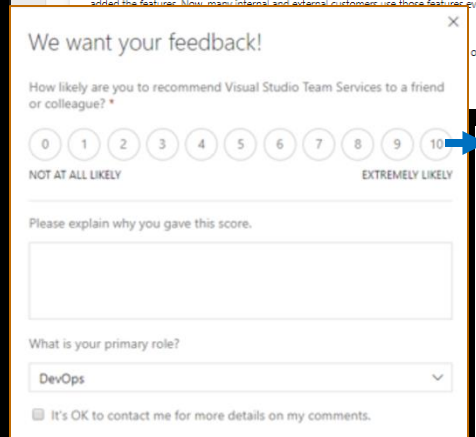
# Take a zero-distance approach by listening to our customers

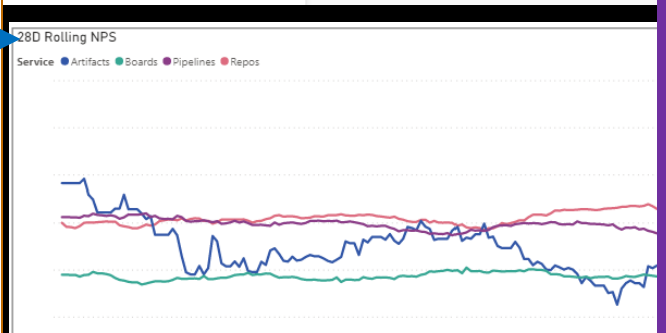
Gathering customer feedback early and often is key to building empathy between engineers and customers.




Direct feedback in product, visible on public site, and captured in backlog

User asked for feedback
Service computes Net Promoter Score (NPS)

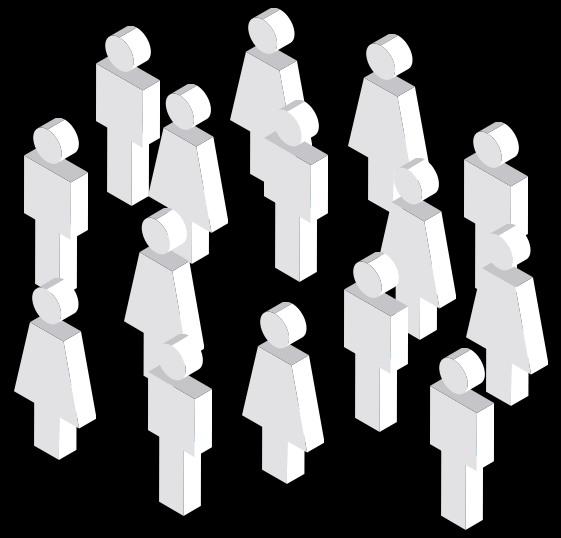
Be customer obsessed

# Ensure products are live in production and collecting feedback

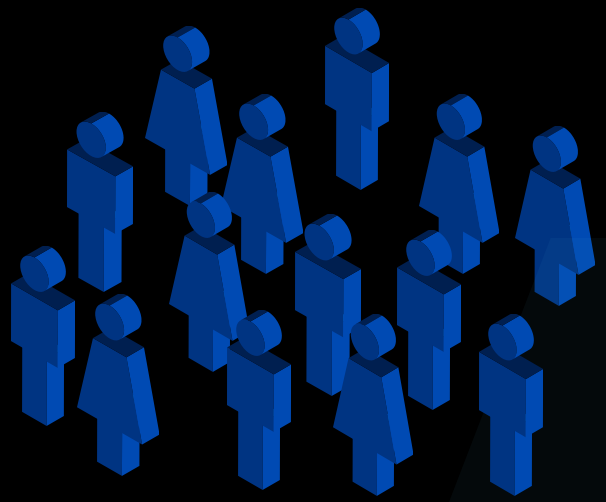
- Collect telemetry data that examines the hypothesis that motivated the deployment.
- Gather information and making incremental changes is key to improvement.
- Treating the goal as a question rather than a statement of fact motivates developers to continue testing their ideas.

Evolve to full stack teams

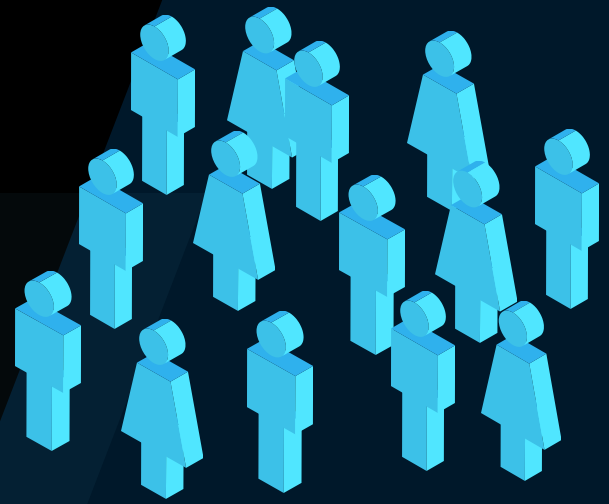
# Evolve the organization



Program management



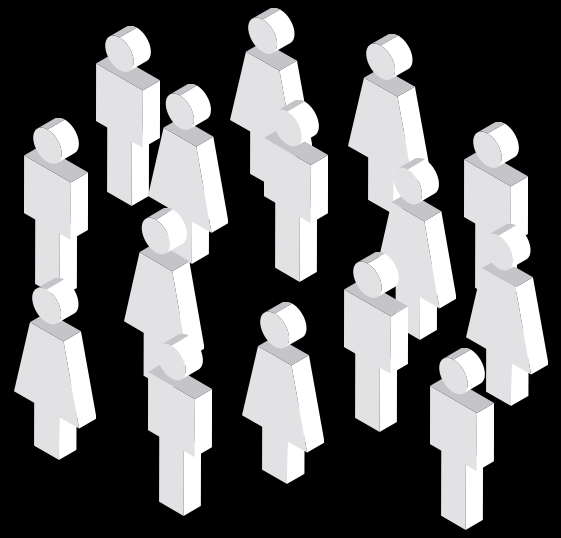
Development



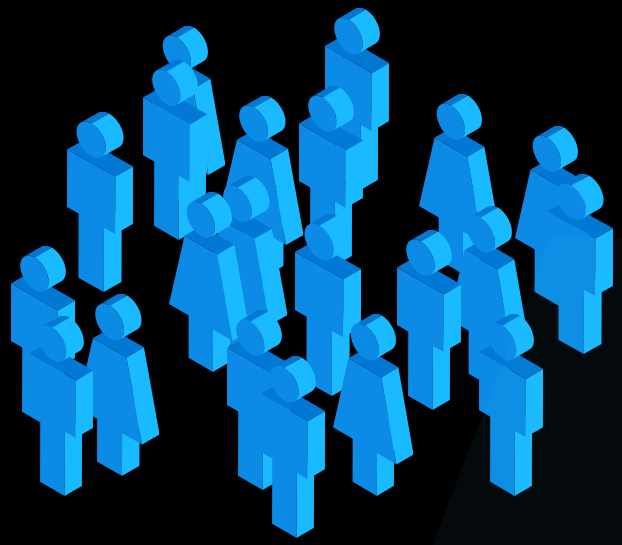
Testing

Evolve to full stack teams

# Evolve the organization



Product management

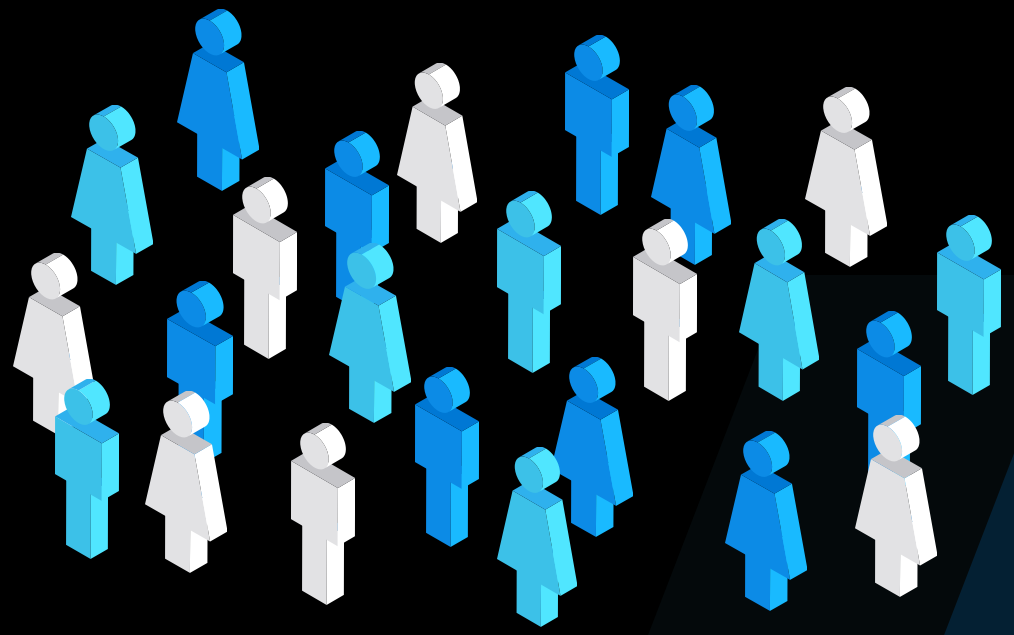


Engineering

Ops/SRE

Evolve to full stack teams

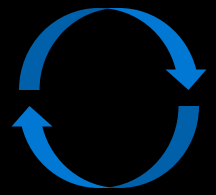
# Evolve the organization



Feature team

## Cultural & Organization Transformation

- Create Clarity
- Be Customer Obsessed
- Evolve Full Stack Teams



## Process & Tools

- { } Collaborate, develop & deliver
- Build for resiliency

# Change how you collaborate, develop, and deliver

Change how you collaborate, develop, and deliver

# Collaborate on code

## Leverage innersource practices to build better products and create a culture of openness by:

Building upon open-source projects that use global contributors

Customizing, adapting, and reusing existing internal code

Collaborating with developers across your organization

Appointing a smaller team to make final decisions

Leveraging open-source software and innersource practices enables your team to innovate and deliver value faster.

Change how you collaborate, develop, and deliver

# Collaborate on code

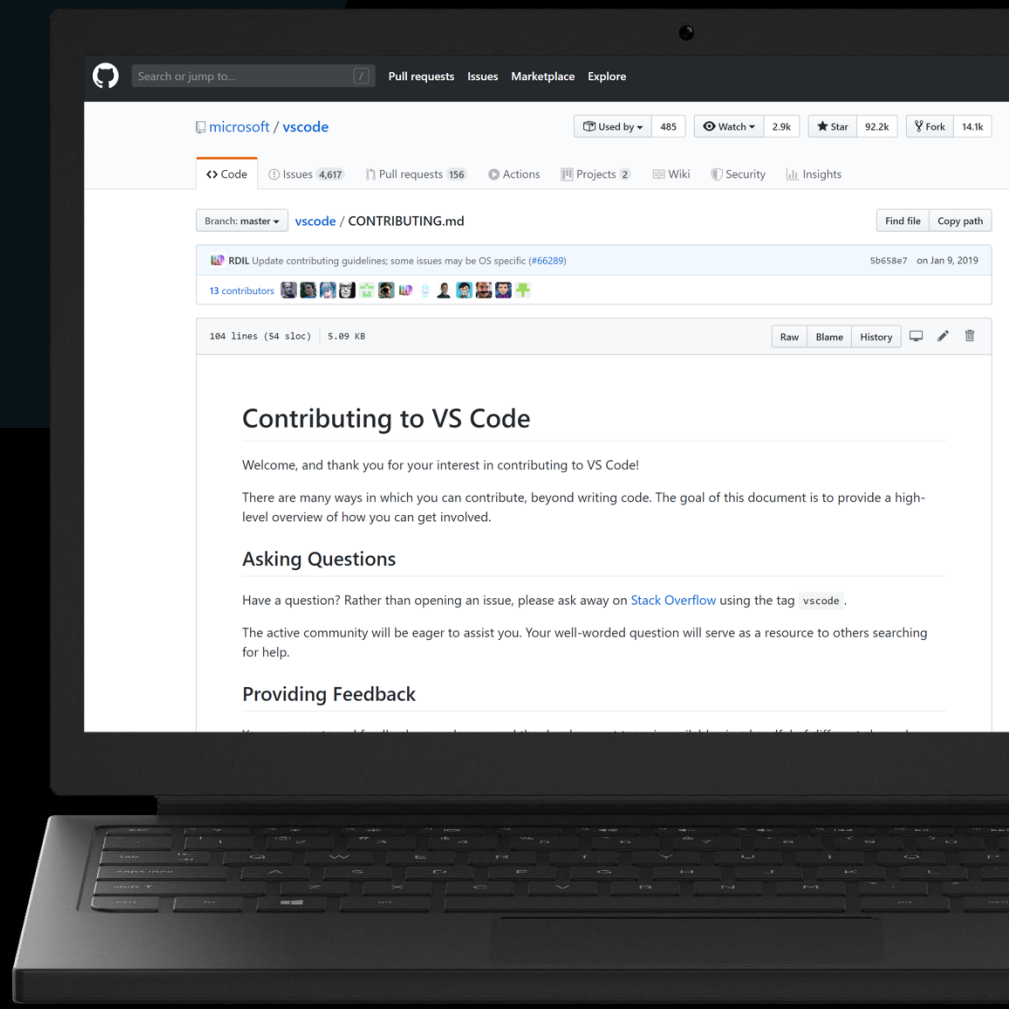
adopt successful open source projects.

## Best practices

Describe the projects intent in README.md.

Describe the contribution process in CONTRIBUTING.md.
Include a call for external contributions.
Contributors don't need special permission.

Maintain and advertise a query for easy first contributions.
Adhere to a reasonable SLA for issues and contributions.
Keep your code clean, commented, and approachable.



# CI/CD pipeline

Automating workflows from code to cloud

## Accelerate delivery through automation

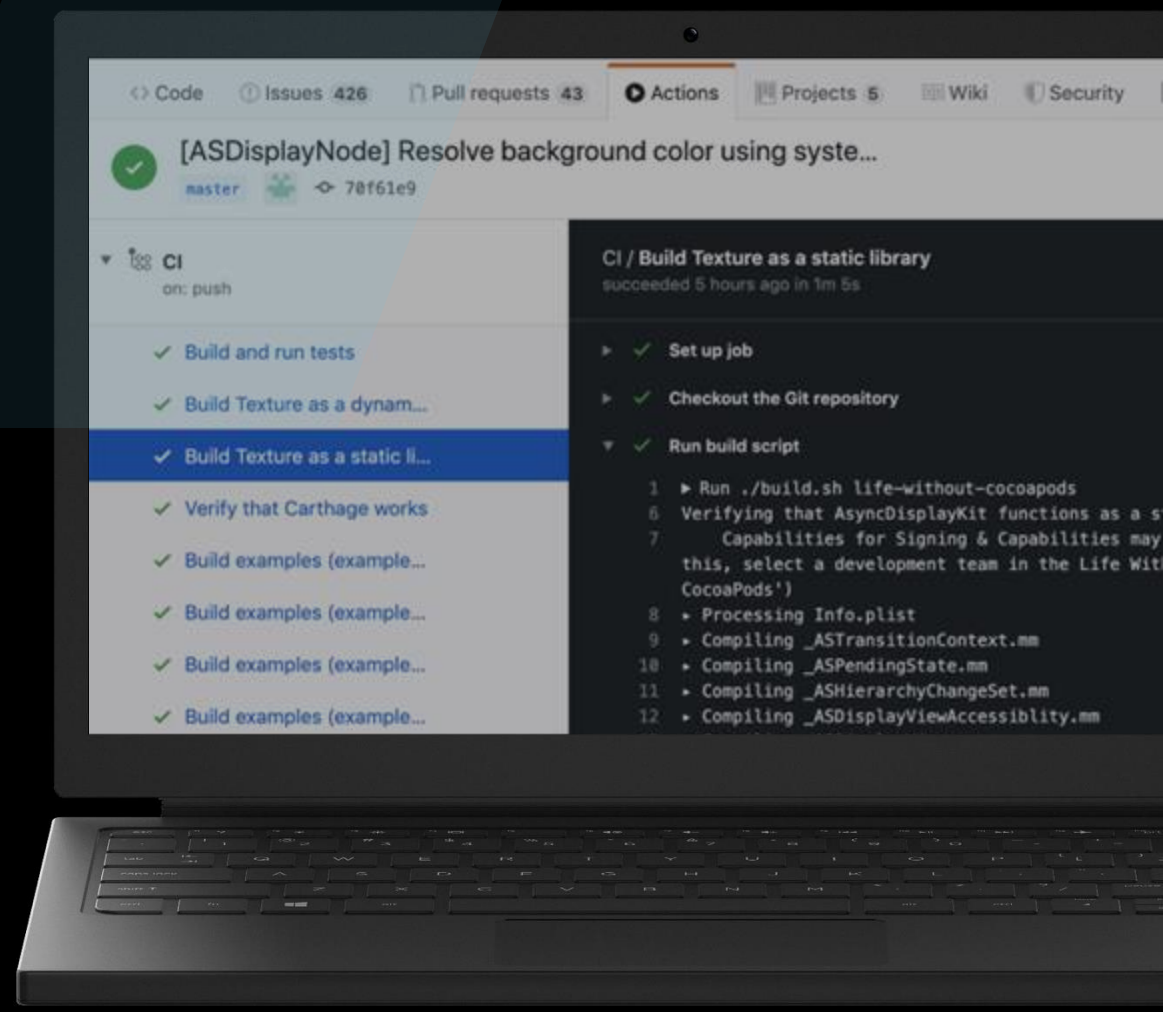
Get access to automation triggers for 20+ project events, which allows for automation beyond just CI/CD to any available API.

## Simple and easy to use

Use configuration based on YAML with a host of sample workflows to learn from and get started.

## Global community for actions

Take advantage of thousands of open-source Actions, maintained by the community and by companies offering integrations, including Microsoft Azure.

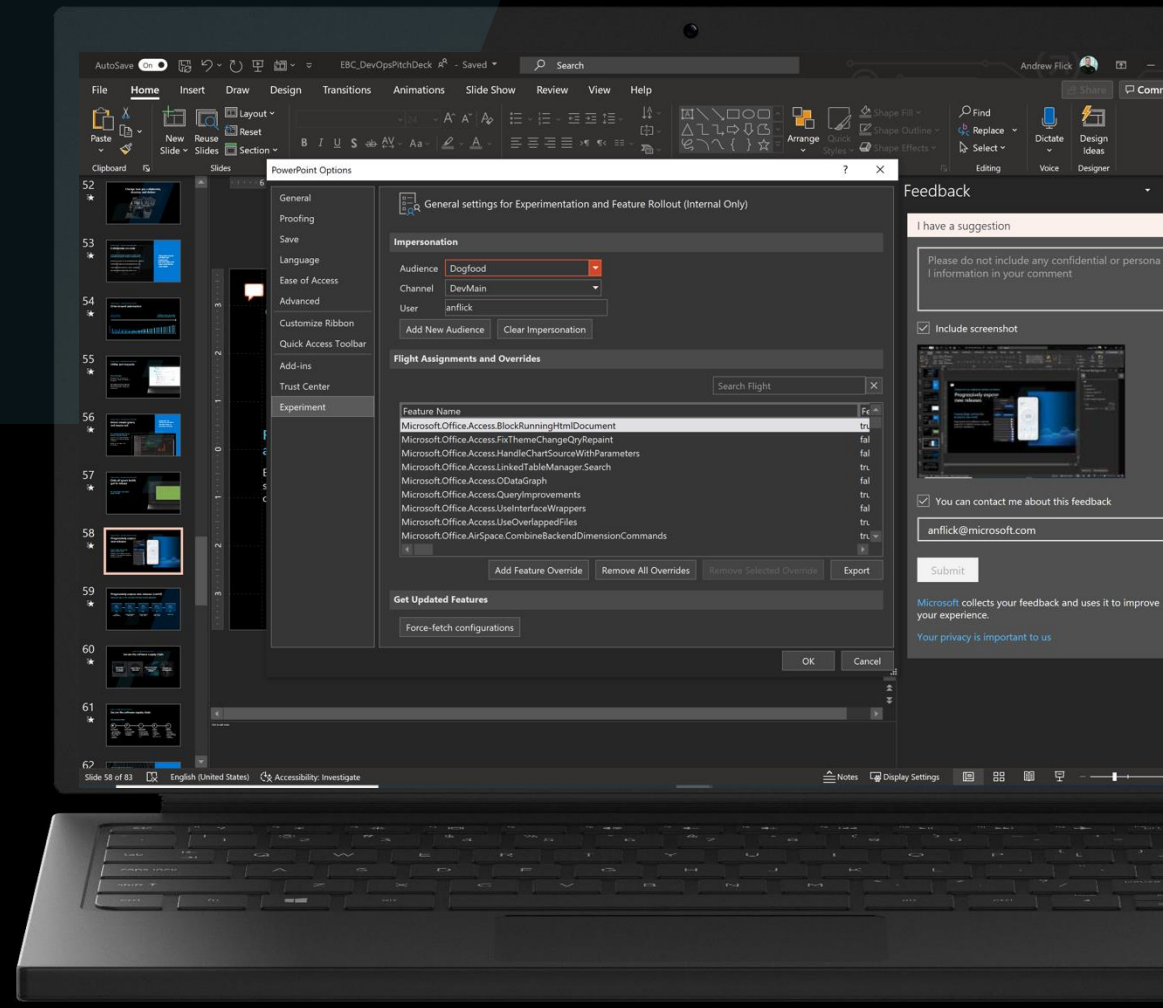


Change how you collaborate, develop, and deliver

# Progressively expose new releases

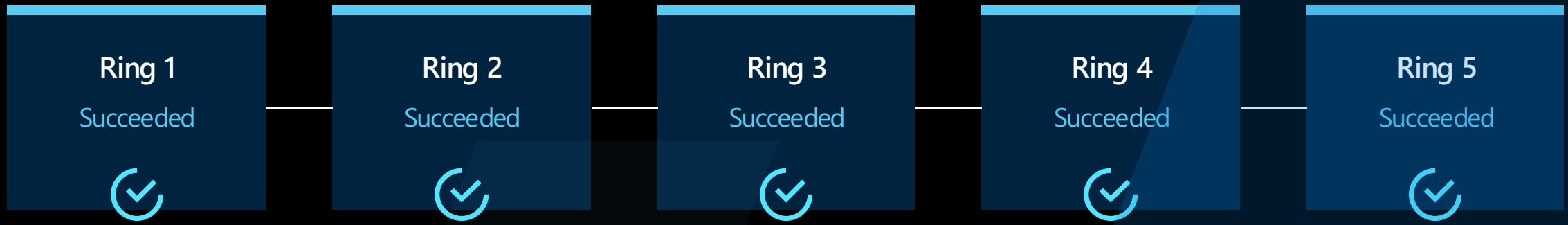
## Feature flags control the access to new work

Experiment across different customer segments to identify feature usage and customer satisfaction.



# Progressively expose new releases (cont'd)

## Deploy one ring at a time to monitor the impact of every deployment.




* not work out when forced to move to Microsoft Stack
  ↳ additive onto Microsoft Stack?
  → the right timing to move

Build for resiliency

# Prioritize reliability and resiliency

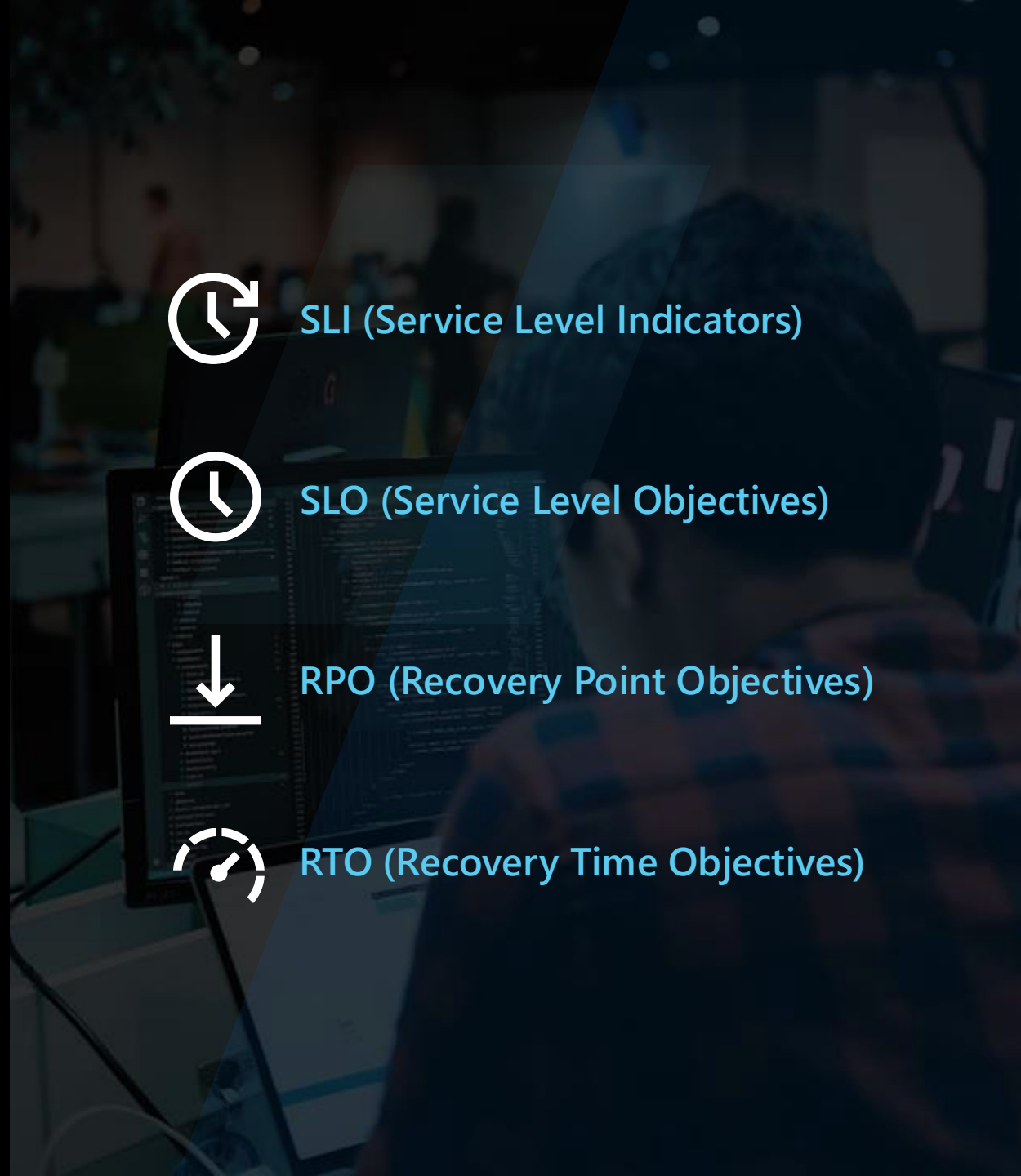
## Reliable apps are:

### Resilient

Able to recover gracefully from failures and continue to function with minimal downtime and data loss

### Highly Available

Able to run as designed in a healthy state with no significant downtime during disruptions



SLI (Service Level Indicators)

SLO (Service Level Objectives)

RPO (Recovery Point Objectives)

RTO (Recovery Time Objectives)

Build for resiliency

# Measure and improve

## Kaizen review

### Incident response by month

- Availability (per customer)
- Automated detection
- Time to detect
- Time to mitigate

### Every incident that affected users

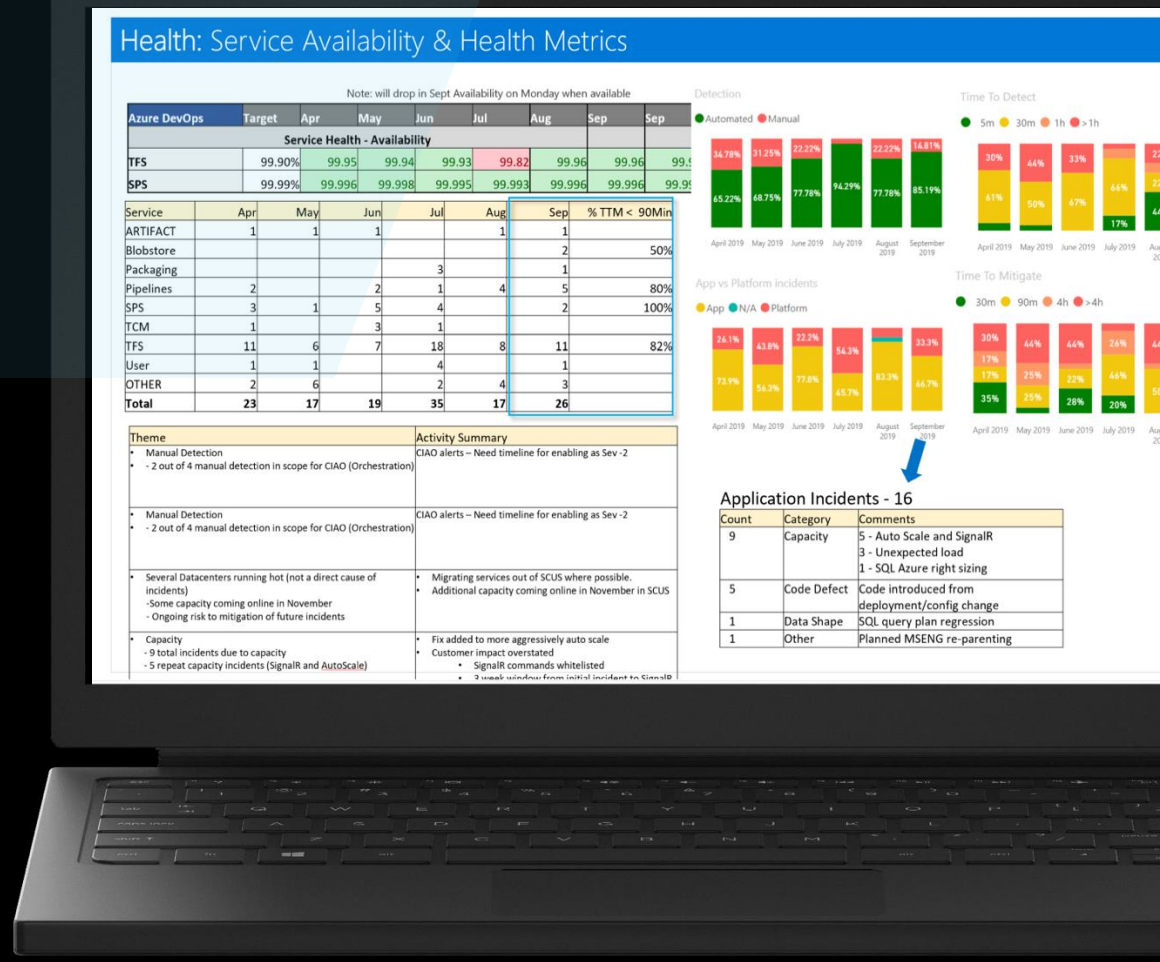
- Identify patterns
- Ensure remediation

### All customer support inquiries

- Reduce incoming/000 users

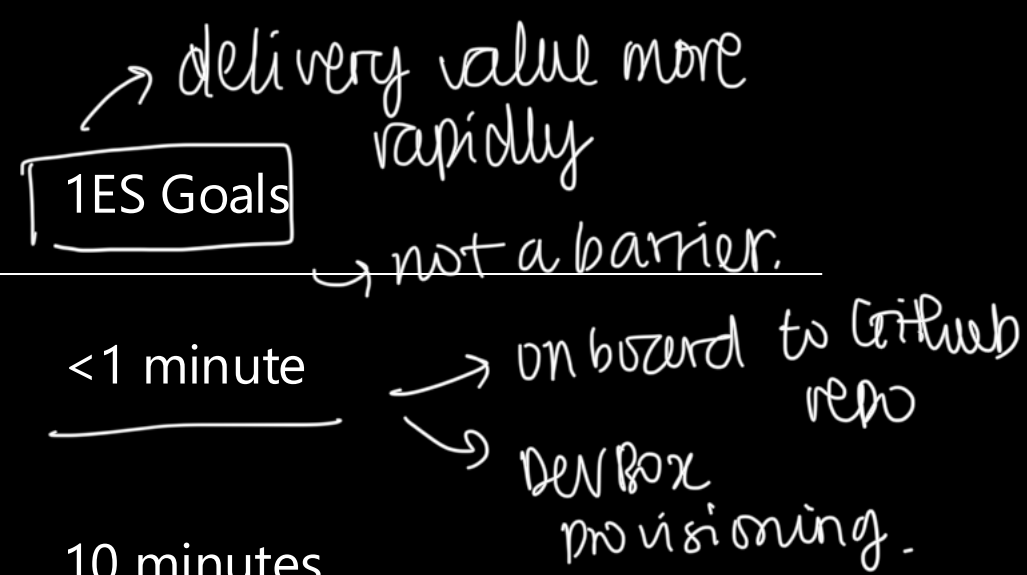
### Cost of operation

- Reduce $/000 users



# Developer Velocity

| | Microsoft today | 1ES Goals |
|---|---|---|
| **Setup Environment** | 2 days | <1 minute |
| **Test Code Change** | 4-8 hours | 10 minutes |
| **Merge New Code** | 2-3 days | 30-60 minutes |
| **Deploy to Production** | 2-3 days | 1-2 hours |
| **Security & Trust** | Continuous | Shift left |



# Microsoft's Internal Engineering Platform

Integrated developer experiences that improve **productivity & satisfaction**

Performance and **security** @ enterprise **scale**

App Developers

Local customizations and Extensibility

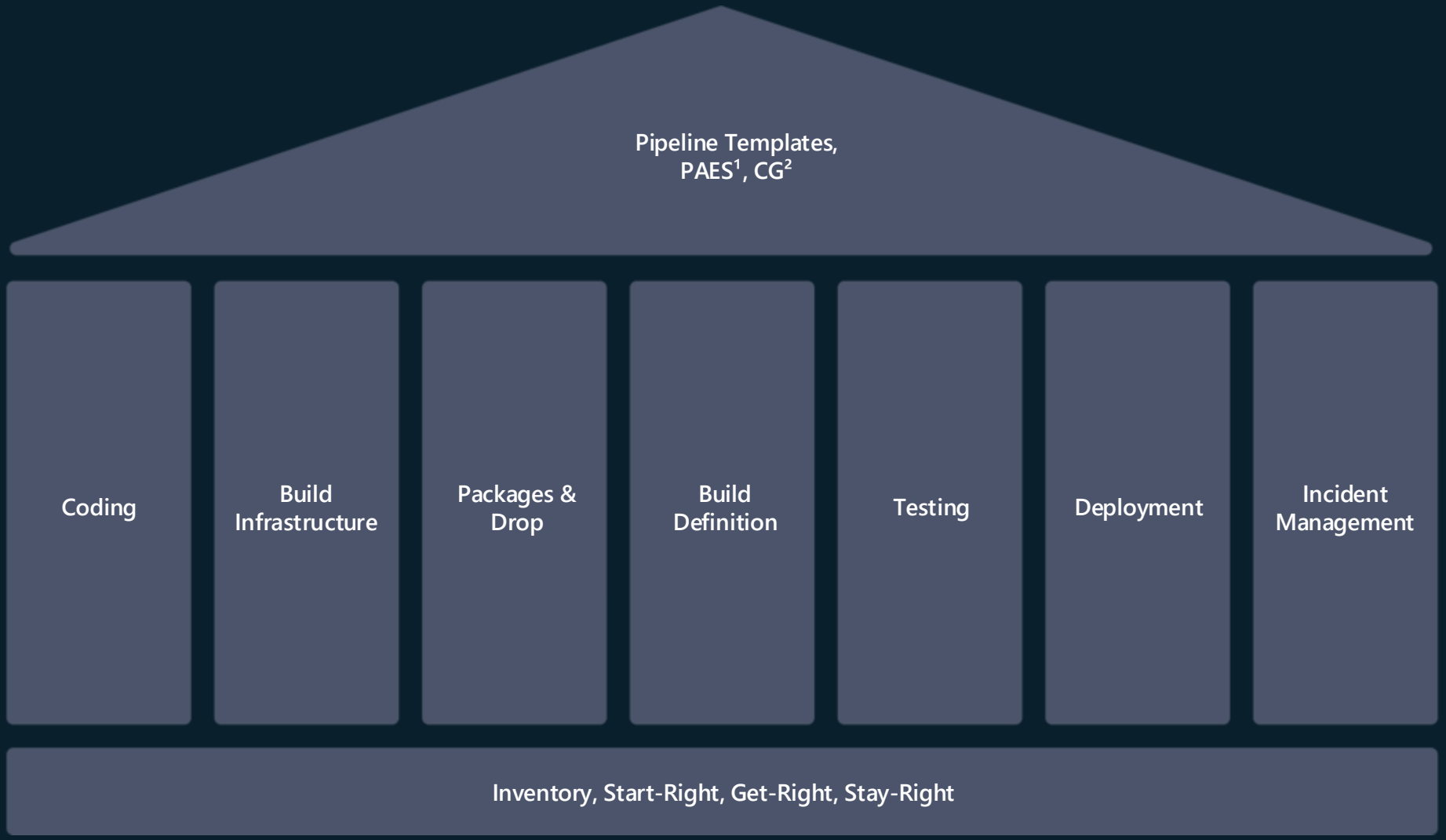
Engineering Platform (1ES)

Security, Platform Management and Integrated Experiences

Microsoft, 3rd party, OSS Developer Products

MICROSOFT CONFIDENTIAL

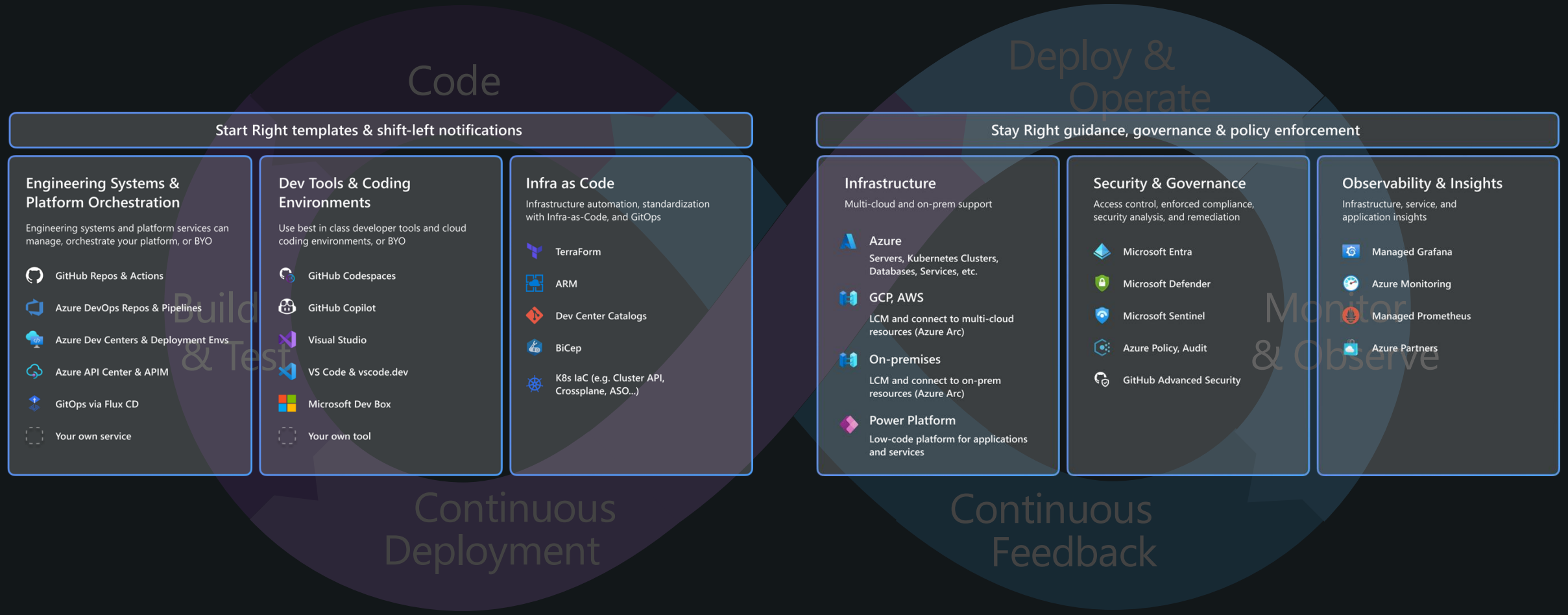
# Engineering Platform: Paved Path




- End to end standard & policy adoption simplification and enforcement
- Process & Tooling standards
- Foundational standards & practices

MICROSOFT CONFIDENTIAL

[1] Policy Aware Engineering System, [2] Component Governance




MICROSOFT CONFIDENTIAL

# *Leverage InnerSource practices to build better products and create a culture of openness*

- Building upon open-source projects that use global contributors
- Customizing, adapting, and reusing existing internal code
- Collaborating with developers across your organization
- Appointing a smaller team to make final decisions

MICROSOFT CONFIDENTIAL

# Our DevOps Transformation – the story so far

## Before

- 4-6 month milestones
- Horizontal teams
- Personal offices
- Long planning cycles
- PM, Dev, Test
- Yearly customer engagement
- Feature branches
- 20+ person teams
- Secret roadmap
- Bug debt
- 100 page spec documents
- Private repositories
- Deep organizational hierarchy
- Success is a measure of install numbers
- Features shipped once a year

## After

- 4-week sprints
- Vertical teams
- Team rooms
- Continual Planning & Learning
- PM & Engineering
- Continual customer engagement
- Everyone in master
- 8-12 person teams
- Publicly shared roadmap
- Zero debt
- Mockups in PPT
- Inner/Open source
- Flattened organization hierarchy
- User satisfaction determines success
- Features shipped every sprint

# Take-a-ways

data could be wrong.

1 Take the science seriously… but don't be overly prescriptive.

2 Stop celebrating activity… start celebrating results.

↳ celebrate the value delivered.

3 You cannot cheat shipping.

4 Build the culture you want… and you'll get the behavior you're after.

Microsoft

# Thank you!

Jay Schmelzer

JaySch@microsoft.com

© Copyright Microsoft Corporation. All rights reserved.